Concurrency Handling Methods (Preventing Double Deduction)

Overview:

Multiple concurrent requests trying to transfer from the same wallet can cause double deduction or race conditions. Here are 3 generic approaches to handle this:

Approach 1: Database Transaction with Row-level Locking

Description:

- Use database transaction with high isolation level (SERIALIZABLE or REPEATABLE_READ)
- Acquire exclusive locks on wallet rows using SELECT FOR UPDATE
- Locks are held until transaction commits/rolls back

Example Flow:

- BEGIN TRANSACTION
- SELECT balance FROM wallet WHERE id = 1 FOR UPDATE (locks row)
- SELECT balance FROM wallet WHERE id = 2 FOR UPDATE (locks row)
- Validate: wallet1.balance >= amount
- UPDATE wallet SET balance = balance - amount WHERE id = 1
- UPDATE wallet SET balance = balance + amount WHERE id = 2
- INSERT INTO transaction (...) VALUES (...)
- COMMIT TRANSACTION

Benefits:

- Database ensures atomicity (all or nothing)
- Prevents concurrent modifications

- Works with any SQL database (PostgreSQL, MySQL, Oracle, etc.)
- Language-independent

Limitations:

- Locks held for entire transaction duration
- Can cause deadlocks if locks are acquired in different order
- May reduce throughput under high concurrency

Approach 2: Optimistic Locking with Version Field

Description:

- Add version/timestamp field to Wallet entity
- Read wallet with current version
- Before update, verify version hasn't changed
- If version changed, transaction fails (retry or reject)

Example Flow:

- READ wallet (id=1, balance=100, version=5)
- Validate: balance >= amount
- UPDATE wallet SET balance = balance - amount, version = version + 1 WHERE id = 1 AND version = 5
- If update affects 0 rows → Version changed → Retry or fail
- If update affects 1 row → Success → Continue with toWallet update

Benefits:

- No long-held locks
- Better for high-read scenarios
- Works with any database and language
- Retry mechanism handles conflicts

Limitations:

- Requires retry logic

- Can fail multiple times before success
- More complex error handling

Approach 3: Distributed Locking (Selected Approach)

Description:

- Use distributed lock manager (Redis, Memcached, etc.)
- Acquire lock on wallet ID before transaction
- Release lock after transaction completes
- Set lock expiration to prevent deadlocks

Example Flow:

- ACQUIRE LOCK("wallet_1", timeout=5 seconds)
    - If lock acquired → Proceed
    - If lock exists → Wait (block) until lock is released or timeout expires
    - If timeout expires → Return error
- READ wallet balance
- Validate: balance >= amount
- UPDATE wallet balances
- CREATE transaction record
- RELEASE LOCK("wallet_1")

Benefits:

- Works across multiple application servers (distributed systems)
- Language and database agnostic
- Prevents concurrent access at application level
- Lock expiration prevents deadlocks
- Handles high concurrency with proper lock management

Limitations:

- Requires external lock service (Redis, etc.)
- Network latency for lock operations
- Need to handle lock expiration and renewal

Selected Approach: Distributed Locking (Approach 3)

The **Digital Wallet system** uses **distributed locking** for the following reasons:

- **Scalability:** Works across multiple application servers
- **Flexibility:** Language and database agnostic
- **Reliability:** Lock expiration prevents deadlocks
- **Performance:** Efficient for high-concurrency scenarios
- **Interview-friendly:** Demonstrates understanding of distributed systems

---

Implementation Details:

- **Lock Service:** Redis or similar distributed cache
- **Lock Key Format:** "wallet_lock_{walletId}"
- **Lock Timeout:** 5 seconds (prevents deadlocks)
- **Lock Acquisition:** Wait with timeout behavior - transaction blocks (waits) until lock is available or timeout expires
- **Lock Behavior:** If lock is taken, transaction waits (retries) up to timeout period; if timeout expires, transaction fails
- **Lock Ordering:** Acquire locks in sorted order (wallet IDs) to prevent deadlocks
- **Lock Release:** Always release in finally block to prevent lock leaks